SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 13

Name of Student : GINNE SAI TEJA
Enrollment No. : 2303A51526
Batch No. : 22

Task Description #1 (Refactoring – Removing Global Variables)
• Task: Use AI to eliminate unnecessary global variables from the code.
• Instructions:
o Identify global variables used across functions.
o Refactor the code to pass values using function parameters.
o Improve modularity and testability.
• Sample Legacy Code:
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
• Expected Output:
o Refactored version passing rate as a parameter or using a configuration structure.

Prompt:-
generate a program to remove unnecessary variable from the below code and print the output to be refactored code that passing rate as a parameter to the function and return the result.

Code:-

```
# Task-1
# generate a program to remove unnecessar
rate = 0.1
def calculate_interest(amount, rate):
    return amount * rate
print(calculate_interest(1000, 0.05))
```

Output:-

```
50.0
PS C:\Users\saite\Downloads>
```

Justification:-
The unnecessary variable `rate` was removed by passing it as a parameter to the function `calculate_interest` and the function now directly returns the calculated interest without relying on an external variable, making the code cleaner and more reusable and maintainable by allowing different interest rates to be used without modifying the function's internal logic.

Task Description #2 : (Refactoring Deeply Nested Conditionals)
• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
• Focus Areas:
o Readability
o Logical simplification
o Maintainability
Legacy Code:
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
Expected Outcome:
o Flattened logic using guard clauses or a mapping-based approach.

Prompt:-
refactor the below code with nested if-elif-else using flattend logic simplification , guard clauses into a cleaner and more readable and maintainable version of the code.

Code:-

```
# Task-2
# refactor the below code with nes
score = 78
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
```

Output:-

```
PS C:\Users\saite\Downloads>  c:; c
xtensions\ms-python.debugpy-2025.18
Very Good
PS C:\Users\saite\Downloads>
```

Justification:-
The original code uses nested if-elif-else statements to evaluate the score and print the corresponding message. The refactored code simplifies the logic by using guard clauses, which allows for a more straightforward and readable structure. Each condition is evaluated independently, and the appropriate message is printed without the need for nested blocks, improving maintainability and readability.

Task 3 (Refactoring Repeated File Handling Code)
• Task: Use AI to refactor repeated file open/read/close logic.
• Focus Areas:
o DRY principle
o Context managers
o Function reuse

Legacy Code:
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
Expected Outcome:
o Reusable function using with open() and parameters.

Prompt:-
refactor the below code with repeated file handling logic that can open a file, read its contents, and close the file after reading, foucus on dry principle and context manager to ensure proper resource management and function reuse and the result of code should be reuseable function using with loop() and parameters.

Code:-

```
# Task-3
# refactor the below code with repeated file ha
f = open("C:\\AIAC LAB\\random.txt")
print(f.read())
f.close()
f = open("C:\\AIAC LAB\\random1.txt")
print(f.read())
f.close()
def read_file(file_name):
    with open(file_name, 'r') as f:
        return f.read()
print(read_file("C:\\AIAC LAB\\random.txt"))
print(read_file("C:\\AIAC LAB\\random1.txt"))
```

Output:

```
# Task-3
# refactor the below code with repeated file ha
f = open("C:\\AIAC LAB\\random.txt")
print(f.read())
f.close()
f = open("C:\\AIAC LAB\\random1.txt")
print(f.read())
f.close()
def read_file(file_name):
    with open(file_name, 'r') as f:
        return f.read()
print(read_file("C:\\AIAC LAB\\random.txt"))
print(read_file("C:\\AIAC LAB\\random1.txt"))
```

Justification:-
The original code contains repeated file handling logic for opening, reading, and closing files. The refactored code defines a reusable function `read_file` that takes a file name as a parameter and uses a context manager (`with` statement) to ensure proper resource management. This eliminates the need for manual file closing and allows for cleaner and more maintainable code by adhering to the DRY (Don't Repeat Yourself) principle. The function can be easily reused for reading any file without duplicating the file handling logic.

Task 4 (Optimizing Search Logic)
• Task: Refactor inefficient linear searches using appropriate data structures.
• Focus Areas:
o Time complexity
o Data structure choice

Legacy Code:
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
Expected Outcome:
o Use of sets or dictionaries with complexity justification.

Prompt:-
refactor the below code with inefficient linear search algorithm using appropriate data structures, show time complexity and algorithms to optimize the search process and improve performance .

Code:-

```
# Task-4
# refactor the below code with inefficient linear sear
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
        break
print("Access Granted" if found else "Access Denied")
```

Output:-

```
n.debugpy-2025.18.0-win32-x64\bundle
Enter username: sai
Access Denied
PS C:\Users\saite\Downloads>
```

Justification:-
The original code uses a linear search algorithm to check if the entered username exists in the list of users, which has a time complexity of O(n). The refactored code can be optimized by using a set data structure, which allows for O(1) average time complexity for membership testing. By converting the list of users into a set, we can significantly improve the performance of the search process, especially as the number of users increases. This optimization enhances the efficiency and scalability of the code while maintaining its functionality.

Task 5 (Refactoring Procedural Code into OOP Design)
• Task: Use AI to refactor procedural code into a class-based design.
• Focus Areas:
o Object-Oriented principles
o Encapsulation

Legacy Code:
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Prompt:-
refactor the below code with procedural programming code in to object-oriented programming codeand Encapsulation to create a class that represents a user and encapsulates the user-related data and behaviors, such as username and access control, to improve code organization and maintainability.

Code:-

```
# Task-5
# refactor the below code with procedural
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
class User:
    def __init__(self, username, salary):
        self.username = username
        self.salary = salary

    def calculate_net_salary(self):
        tax = self.salary * 0.2
        net_salary = self.salary - tax
        return net_salary
user = User("John", 50000)
print(user.calculate_net_salary())
```

Output:-

```
.py
40000.0
40000.0
PS C:\Users\saite\Downloads>
```

Justification:-
The original code is procedural and calculates the net salary by directly manipulating variables. The refactored code defines a `User` class that encapsulates the user-related data (username and salary) and behaviors (calculating net salary). This object-oriented approach improves code organization by grouping related data and functionality together, making it easier to maintain and extend in the future. The `calculate_net_salary` method allows for better encapsulation of the logic related to salary calculation, enhancing code readability and maintainability.

Task 6 (Refactoring for Performance Optimization)
• Task: Use AI to refactor a performance-heavy loop handling large data.
• Focus Areas:
o Algorithmic optimization
o Use of built-in functions

Legacy Code:
total = 0
for i in range(1, 1000000):

if i % 2 == 0:
total += i
print(total)
Expected Outcome:
o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Prompt:-
refactor the below code with heavy performance loop handling and having large data sets using list comprehension and generator expressions to optimize memory usage and improve performance when processing large data sets and show algorithmic optimization , use of built in functions.

Code:-

```
# Task-6
# refactor the below code with heavy performance loop han
total = sum(i for i in range(1, 1000000) if i % 2 == 0)
print(total)
```

Output:-

```
nt13.py'
249999500000
PS C:\Users\saite\Downloads>
```

Justification:-
The original code uses a generator expression to calculate the sum of even numbers from 1 to 999,999, which is memory efficient. However, we can further optimize the performance by using a mathematical formula to calculate the sum of even numbers directly without iterating through all the numbers. The sum of the first n even numbers can be calculated using the formula n * (n + 1), where n is the number of even numbers. In this case, there are 499,999 even numbers between 1 and 999,999, so we can directly calculate the total as follows.

Task 7 (Removing Hidden Side Effects)
• Task: Refactor code that modifies shared mutable state.
• Focus Areas:
o Functional-style refactoring
o Predictability

Legacy Code:
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)
print(data)
Expected Outcome:
o Refactored function returning values instead of mutating globals.

Prompt:-
refactor the below code with modified shared mutable state, and functional style using refactoring functions returning new values instead of modifying shared mutable state to avoid unintended side effects and improve code reliability and maintainability.

Code:-

```
# Task-7
# refactor the below code w
def add_item(data, x):
    return data + [x]
data = [50]
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

Output:-

```
.py'
[50, 10, 20]
PS C:\Users\saite\Downloads>
```

Justification:-
The original code modifies the shared mutable state (the `data` list) by appending new items to it. The refactored code defines a function `add_item` that takes the existing data and an item to add, and returns a new list with the item added, rather than modifying the original list. This functional style avoids unintended side effects and improves code reliability and maintainability by ensuring that the original data remains unchanged, making it easier to reason about the code and reducing the risk of bugs caused by shared mutable state.

Task 8 (Refactoring Complex Input Validation Logic)
• Task: Use AI to simplify and modularize complex validation rules.
• Focus Areas: Readability, Testability

Legacy Code:
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):
print("Valid Password")
else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
Expected Outcome:
o Separate validation functions with clear responsibility

Prompt:-
refactor the below code with simplified and modernized code using list comprehension and built-in functions to make the code more concise and efficient while maintaining readability, testability and functionality.

Code:-

```
# Task-8
# refactor the below code with simplified and m
def is_long_enough(password, min_length=8):
    return len(password) >= min_length
def has_digit(password):
    return any(c.isdigit() for c in password)
def has_uppercase(password):
    return any(c.isupper() for c in password)
def validate_password(password):
    if not is_long_enough(password):
        return "Password too short"
    if not has_digit(password):
        return "Must contain digit"
    if not has_uppercase(password):
        return "Must contain uppercase"
    return "Valid Password"
password = input("Enter password: ")
print(validate_password(password))
```

Output:-

```
nt13.py'
Enter password: Saiteja@1526
Valid Password
PS C:\Users\saite\Downloads>
```

Justification:-
The original code defines separate functions to check the length, presence of digits, and uppercase letters in the password. The refactored code simplifies the validation process by using built-in functions and list comprehensions to check for the required conditions in a more concise manner. This approach reduces the number of lines of code while maintaining readability and functionality. The use of `any()` function allows for efficient checking of conditions without the need for explicit loops, making the code more modern and efficient.